

3F7 Information Theory and Coding

Full Technical Report

Liu Zihe
zl559

University of Cambridge — 6 December 2025

1 Introduction

Brief intro of whole Report

2 Overview of Short Lab

summary of the lessons learned in the short lab (this should be fairly short, e.g., one page, and stick to the essentials)

refer to 3F7/3F7lab.ipynb for this

3 Coding for Discrete Stationary Sources

3.1 Background: Entropy of Stochastic Processes

Explain sources with memory, stationary sources etc

Explain core math behind section 1.1 in the handout. Refer to cover and thomas elements of information theory chapter 4 (should be attached and fed into your context window). Include stationarity and markov chains and their associated entropy math.

Note to self: memoryless means iid. Discrete stationary process can (and usually does have memory). Conditional probabilities depend only on relative positions (shift invariant). We are using strict stationarity here btw.

3.1.1 Theorem 1

proofs for all 4 statements in theorem1. For statement4, take the simplest proof. I have provided some context from cover and thomas as reference; you don't have to follow their method

3.1.2 Theorem 2

3.2 Coding Schemes for Discrete Stationary Processes

think about what this implies in terms of the coding techniques you've learned during the short lab.

How would you implement Huffman and arithmetic coding for discrete stationary sources to achieve a good compression rate close to the entropy rate of a discrete stationary source?

Can you think of a practical compression technique for which n can be made very large and hence have performance approaching the entropy rate, and how would this work in practice?

3.3 Entropy Rates of Discrete Stationary Processes

For measuring the entropy rate, take the text file hamlet.txt and try to measure the entropy resulting from estimating the probabilities of couples, triples, quadruples etc. of consecutive symbols in the text file. We've provided a Jupyter notebook to help you measure these

You will observe that while your estimates of H_1 , H_2 and possibly H_3 are fairly accurate, the estimated entropies keep decreasing with apparently no positive lower bound, implying possibly that $H_{\text{inf}} = 0$.

Do you really believe that the entropy rate of text is zero? What do you think may be going wrong in this experiment?

4 Adaptive Coding

4.1 Background: Universal Codes

Using section 13 of Cover and Thomas, explain the math background behind this stuff:

The problem of probability estimation is even more interesting when you consider that a good compression algorithm will in fact not need a separate stored probability model. Compression can proceed starting from a startup probability model that assigns for example a uniform distribution to all symbols, and adapt its model “on the fly” as you progress through the file. As long as you always encode a symbol using the “old” model before updating your estimated probabilities to get a “new” model, the decoder will be able to decode the source symbol and update the source model to mirror exactly the updates done by the encoder.

Prove this estimator stuff:

In IB Paper 7, you've learned about the Beta distribution (Lecture slides 3) for estimating the unknown parameter of a source of Bernoulli random variables. If p is the unknown Bernoulli parameter $p = P(X=1)$ of a binary source, by assuming that p is a random variable P uniformly distributed between zero and one (“uniform prior”), then the distribution of P conditioned on the data if you've observed n_0 zeros and n_1 ones is a Beta distribution $\text{Beta}(n_0, n_1)$. The Laplace estimator is the expected value of the Beta distribution $E[P | X_1, \dots, X_n]$ where $X_1, \dots, X_n$ is the data seen after $n = n_0 + n_1$ observations $E[P | X_1, \dots, X_k] = \frac{n_1 + 1}{n_0 + n_1 + 2}$, which is the same as saying that before you start observing source outputs, you assume that you've already seen one 0 and one 1. You can prove all of the above if you feel adventurous.

4.2 Adaptive Arithmetic Coding

Without actually implementing an adaptive arithmetic code, you can get a good estimate of its performance by assuming that the length of the code sequence for a string of data $x_1, x_2, \dots, x_n$ whose estimated probability is

Based on this estimate, you can investigate and answer the following questions by conducting a few statistical measurements on the file hamlet.txt:

4.2.1 Effect of length n on performance

how does the performance of adaptive arithmetic coding approach the entropy when probabilities are adapted at each step using the estimator in (3) in function of the data length n ? You could plot L/n calculating L based on (4) using the first n symbols in Hamlet, for n going

from one to, say, 10,000, using, say $\delta = 1$ (as in the Laplace estimator). Does the resulting performance tend to the entropy of the distribution?

4.2.2 Effect of bias δ on performance

How does the bias δ affect the performance of the compression algorithm? Try various values for δ and give an indication of how δ is best picked. If you're feeling adventurous, you could try to search for the optimal δ for a few compressed files using a Python search library.

4.2.3 Adaptive and non-adaptive coding schemes

Of course, there may be some scenarios where storage of such a model can indeed be discounted, for example if compressing many files with a unique probability model that has been measured offline and can be viewed as "part of the software package" rather than stored specifically for each file as we have done in the short lab.

For other scenarios, you could also compare the performance of the two while accounting for the fixed extra storage need for the offline model.

What are your conclusions from this comparison?

Is there a "sweet spot" for the file length from when it becomes more advantageous to store the model offline than to measure it on the fly?

5 Adaptive Coding for Discrete Stationary Processes

How does adaptive probability modelling apply to sources with memory?

What are the advantages and drawbacks of considering ever longer block lengths or context lengths, if you are working with estimated probabilities rather than assuming that probabilities are just given to you and accurate as we assumed in the previous section?

6 Conclusion

Some guidelines for you (the LLM):

- Please ignore the section on rate distortion theory, this is wholly unnecessary.
- Whenever you reference proofs, *please* add a reference to the relevant section of the guide (e.g. cover and thomas)
- Please do not supply any python code to this FTR report, unless it is core pseudocode (please notify me if you plan to do this)